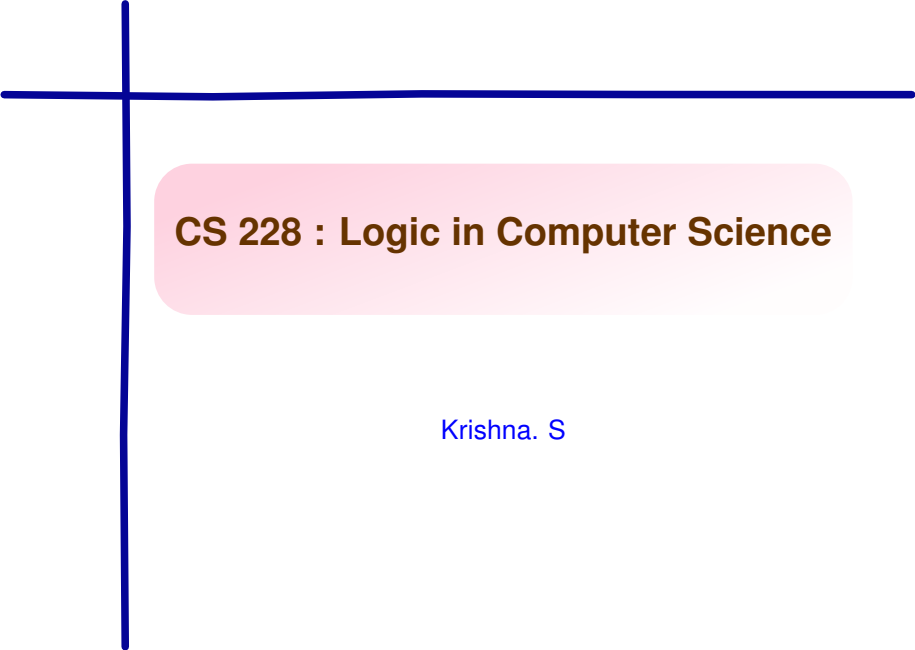


A thick blue line forming an L-shape, with a vertical segment on the left and a horizontal segment at the top, intersecting at the top-left corner of the slide.

CS 228 : Logic in Computer Science

Krishna. S

Recap

- ▶ FOL over words : Satisfiability
- ▶ Translation from formulae φ to *equivalent DFA* A_φ , that is, $L(\varphi) = L(A_\varphi)$
- ▶ For a DFA A , what is $L(A)$?
- ▶ For a FO formula on the word signature, what is $L(\varphi)$?

Recap

- ▶ FOL over words : Satisfiability
- ▶ Translation from formulae φ to *equivalent DFA* A_φ , that is, $L(\varphi) = L(A_\varphi)$
- ▶ For a DFA A , what is $L(A)$?
- ▶ For a FO formula on the word signature, what is $L(\varphi)$?
 - ▶ We defined in l15-16, that

$$L(\varphi) = \{w \mid w \models \varphi\}$$

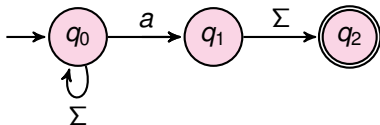
for a sentence φ

- ▶ Recall how we defined satisfaction of formulae in general

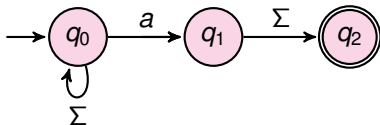
Recap

- ▶ Proof of $L(\varphi) = L(A_\varphi)$ by structural induction
- ▶ $\neg, \wedge, \vee$ mapping to complementation, intersection and union
 - ▶ Union in DFA $\rightarrow$ disjunction in logic
 - ▶ Intersection in DFA $\rightarrow$ conjunction in logic
 - ▶ Complementation in DFA $\rightarrow$ Negation in logic
- ▶ How to handle quantifiers?

Non-determinism



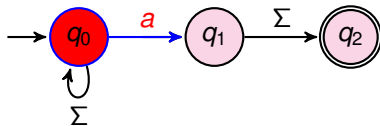
Non-determinism



- ▶ Assume we relax the condition on transitions, and allow
 - ▶ $\delta : Q \times \Sigma \rightarrow 2^Q$
 - ▶ $\delta(q_0, a) = \{q_0, q_1\}, \delta(q_2, a) = \delta(q_2, b) = \emptyset$
 - ▶ Is *aabb* accepted?

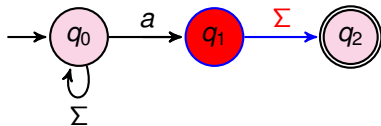
One run of *aabb*

Is *aabb* accepted?



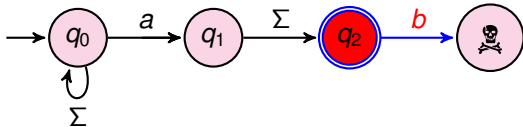
One run of *aabb*

Is *aabb* accepted?



One run of *aabb*

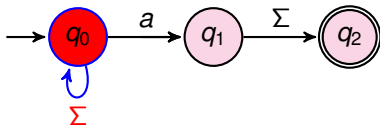
Is *aabb* accepted?



- A non-accepting run for *aabb*

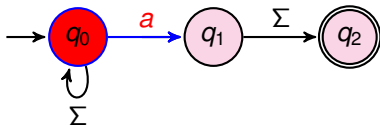
Another run of *aabb*

Is *aabb* accepted?



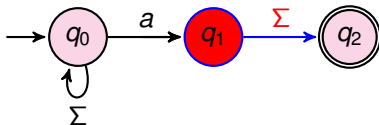
Another run of *aabb*

Is *aabb* accepted?



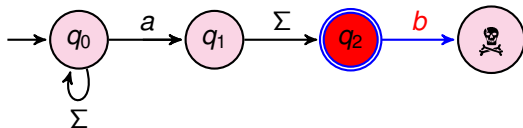
Another run of *aabb*

Is *aabb* accepted?



Another run of *aabb*

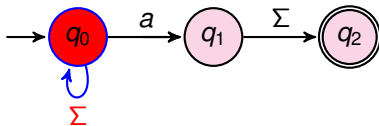
Is *aab****b*** accepted?



- ▶ A non-accepting run for *aabb*

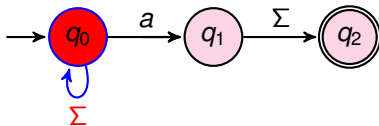
Another run of *aaab*

Is *aaab* accepted?



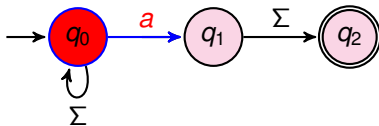
Another run of *aaab*

Is *a***a***ab* accepted?



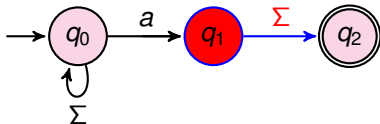
Another run of *aaab*

Is *aaab* accepted?



Another run of *aaab*

Is *aaab* accepted?

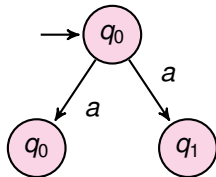


- An accepting run for *aaab*

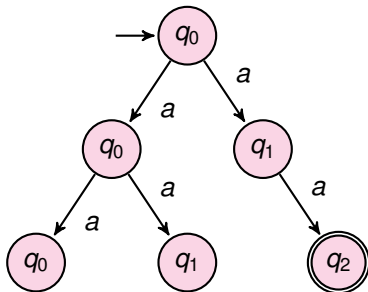
Nondeterministic Finite Automata(NFA)

- ▶ $N = (Q, \Sigma, \delta, Q_0, F)$
 - ▶ Q is a finite set of states
 - ▶ $Q_0 \subseteq Q$ is the set of initial states
 - ▶ $\delta : Q \times \Sigma \rightarrow 2^Q$ is the transition function
 - ▶ $F \subseteq Q$ is the set of final states
- ▶ Acceptance condition : A word w is accepted iff it has atleast one accepting path

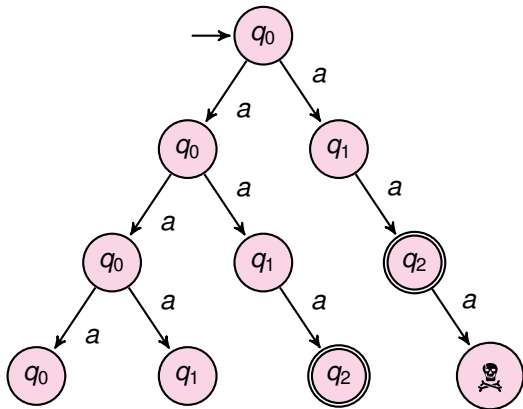
Run Tree of *aaab*



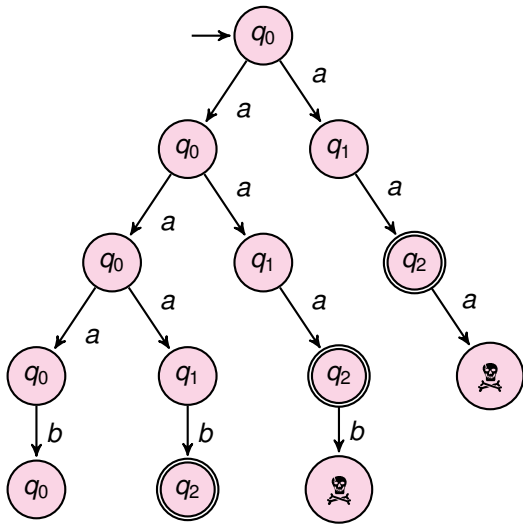
Run Tree of *aaab*



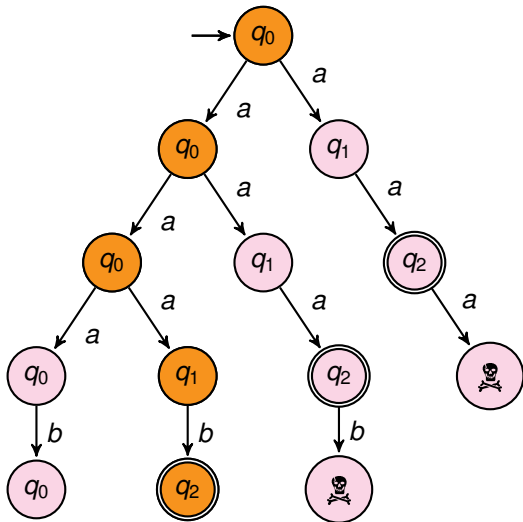
Run Tree of *aaab*



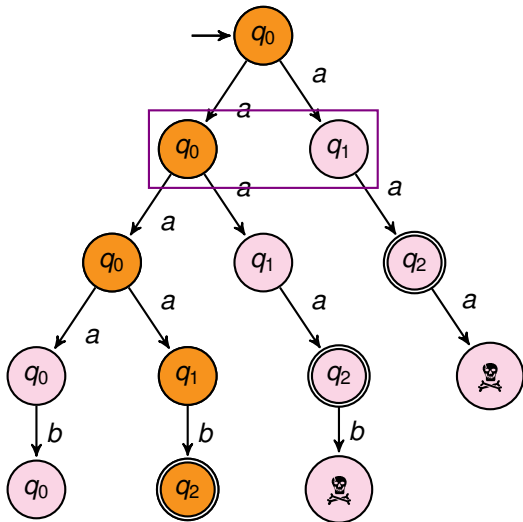
Run Tree of *aaab*



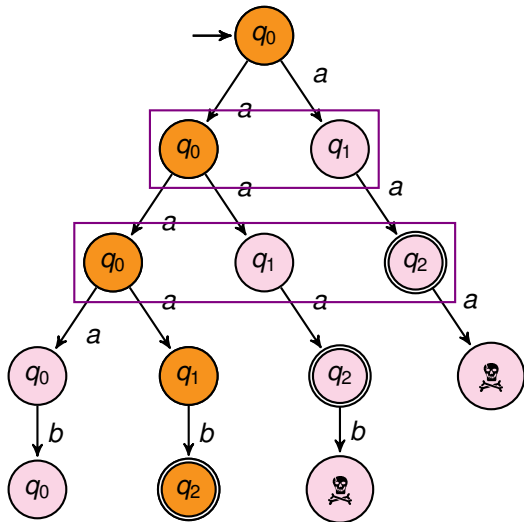
Run Tree of *aaab*



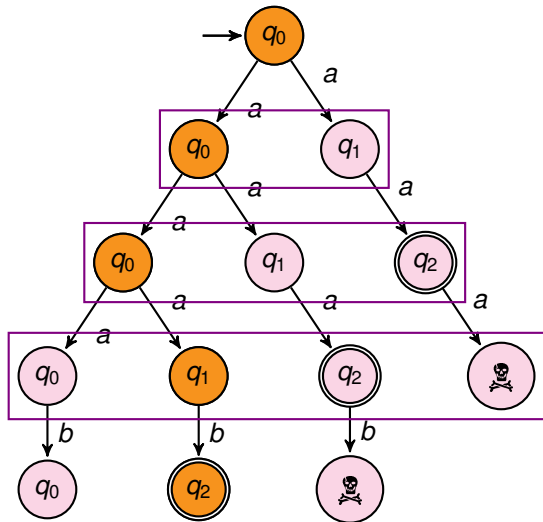
Run Tree of *aaab*



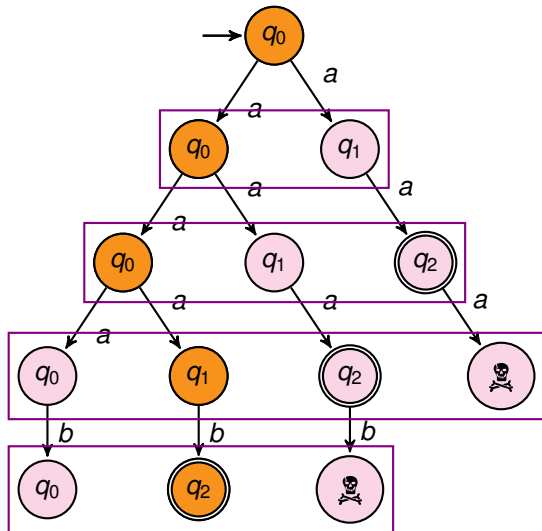
Run Tree of *aaab*



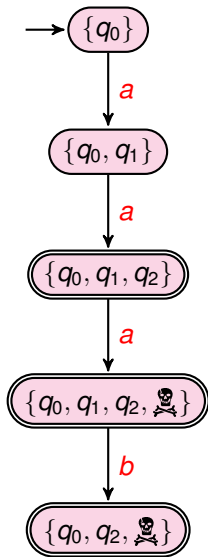
Run Tree of *aaab*



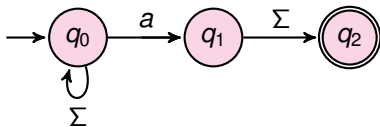
Run Tree of *aaab*



The Single Run



NFA to DFA : On the board



NFA and DFA

- ▶ Any DFA is also an NFA

NFA and DFA

- ▶ Any DFA is also an NFA
- ▶ Any NFA can be converted into a language equivalent DFA

NFA and DFA

- ▶ Any DFA is also an NFA
- ▶ Any NFA can be converted into a language equivalent DFA
 - ▶ Combine all the runs of w in the NFA into a single run in the DFA

NFA and DFA

- ▶ Any DFA is also an NFA
- ▶ Any NFA can be converted into a language equivalent DFA
 - ▶ Combine all the runs of w in the NFA into a single run in the DFA
 - ▶ Combine states occurring in various runs to obtain a set of states

NFA and DFA

- ▶ Any DFA is also an NFA
- ▶ Any NFA can be converted into a language equivalent DFA
 - ▶ Combine all the runs of w in the NFA into a single run in the DFA
 - ▶ Combine states occurring in various runs to obtain a set of states
 - ▶ A set of states evolves into another set of states

NFA and DFA

- ▶ Any DFA is also an NFA
- ▶ Any NFA can be converted into a language equivalent DFA
 - ▶ Combine all the runs of w in the NFA into a single run in the DFA
 - ▶ Combine states occurring in various runs to obtain a set of states
 - ▶ A set of states evolves into another set of states
 - ▶ Use $\delta : Q \times \Sigma \rightarrow 2^Q$, obtain $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$

NFA and DFA

- ▶ Any DFA is also an NFA
- ▶ Any NFA can be converted into a language equivalent DFA
 - ▶ Combine all the runs of w in the NFA into a single run in the DFA
 - ▶ Combine states occurring in various runs to obtain a set of states
 - ▶ A set of states evolves into another set of states
 - ▶ Use $\delta : Q \times \Sigma \rightarrow 2^Q$, obtain $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$
 - ▶ Δ is an extension of δ

NFA and DFA

- ▶ Any DFA is also an NFA
- ▶ Any NFA can be converted into a language equivalent DFA
 - ▶ Combine all the runs of w in the NFA into a single run in the DFA
 - ▶ Combine states occurring in various runs to obtain a set of states
 - ▶ A set of states evolves into another set of states
 - ▶ Use $\delta : Q \times \Sigma \rightarrow 2^Q$, obtain $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$
 - ▶ Δ is an extension of δ
 - ▶ Accept if the obtained set of states contains a final state

NFA and DFA

Given NFA $N = (Q, \Sigma, Q_0, \delta, F)$, obtain the DFA $D = (2^Q, \Sigma, Q_0, \Delta, F')$

NFA and DFA

Given NFA $N = (Q, \Sigma, Q_0, \delta, F)$, obtain the DFA $D = (2^Q, \Sigma, Q_0, \Delta, F')$

- ▶ $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$ is defined by $\Delta(A, a) = \bigcup_{q \in A} \delta(q, a)$

NFA and DFA

Given NFA $N = (Q, \Sigma, Q_0, \delta, F)$, obtain the DFA $D = (2^Q, \Sigma, Q_0, \Delta, F')$

- ▶ $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$ is defined by $\Delta(A, a) = \bigcup_{q \in A} \delta(q, a)$
- ▶ $F' = \{S \in 2^Q \mid S \cap F \neq \emptyset\}$

NFA and DFA

Given NFA $N = (Q, \Sigma, Q_0, \delta, F)$, obtain the DFA $D = (2^Q, \Sigma, Q_0, \Delta, F')$

- ▶ $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$ is defined by $\Delta(A, a) = \bigcup_{q \in A} \delta(q, a)$
- ▶ $F' = \{S \in 2^Q \mid S \cap F \neq \emptyset\}$

Note that $\hat{\delta}(A, a) = \bigcup_{q \in A} \delta(q, a) = \Delta(A, a)$

NFA and DFA

Given NFA $N = (Q, \Sigma, Q_0, \delta, F)$, obtain the DFA $D = (2^Q, \Sigma, Q_0, \Delta, F')$

- ▶ $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$ is defined by $\Delta(A, a) = \bigcup_{q \in A} \delta(q, a)$
- ▶ $F' = \{S \in 2^Q \mid S \cap F \neq \emptyset\}$

Note that $\hat{\delta}(A, a) = \bigcup_{q \in A} \delta(q, a) = \Delta(A, a)$

Show that

- ▶ $\hat{\Delta} : 2^Q \times \Sigma^* \rightarrow 2^Q$ is same as $\hat{\delta} : 2^Q \times \Sigma^* \rightarrow 2^Q$ (recall $\delta : Q \times \Sigma \rightarrow 2^Q$)

NFA and DFA

Given NFA $N = (Q, \Sigma, Q_0, \delta, F)$, obtain the DFA $D = (2^Q, \Sigma, Q_0, \Delta, F')$

- ▶ $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$ is defined by $\Delta(A, a) = \bigcup_{q \in A} \delta(q, a)$
- ▶ $F' = \{S \in 2^Q \mid S \cap F \neq \emptyset\}$

Note that $\hat{\delta}(A, a) = \bigcup_{q \in A} \delta(q, a) = \Delta(A, a)$

Show that

- ▶ $\hat{\Delta} : 2^Q \times \Sigma^* \rightarrow 2^Q$ is same as $\hat{\delta} : 2^Q \times \Sigma^* \rightarrow 2^Q$ (recall $\delta : Q \times \Sigma \rightarrow 2^Q$)
- ▶ $\hat{\Delta}(A, xa) = \Delta(\hat{\Delta}(A, x), a) = \bigcup_{q \in \hat{\Delta}(A, x)} \delta(q, a)$

NFA and DFA

Given NFA $N = (Q, \Sigma, Q_0, \delta, F)$, obtain the DFA $D = (2^Q, \Sigma, Q_0, \Delta, F')$

- ▶ $\Delta : 2^Q \times \Sigma \rightarrow 2^Q$ is defined by $\Delta(A, a) = \bigcup_{q \in A} \delta(q, a)$
- ▶ $F' = \{S \in 2^Q \mid S \cap F \neq \emptyset\}$

Note that $\hat{\delta}(A, a) = \bigcup_{q \in A} \delta(q, a) = \Delta(A, a)$

Show that

- ▶ $\hat{\Delta} : 2^Q \times \Sigma^* \rightarrow 2^Q$ is same as $\hat{\delta} : 2^Q \times \Sigma^* \rightarrow 2^Q$ (recall $\delta : Q \times \Sigma \rightarrow 2^Q$)
- ▶ $\hat{\Delta}(A, xa) = \Delta(\hat{\Delta}(A, x), a) = \bigcup_{q \in \hat{\Delta}(A, x)} \delta(q, a)$
- ▶ $\hat{\delta}(A, xa) = \bigcup_{q \in \hat{\delta}(A, x)} \delta(q, a)$

NFA = DFA

$$x \in L(D) \leftrightarrow \hat{\Delta}(Q_0, x) \in F'$$

$$\leftrightarrow$$

$$\hat{\delta}(Q_0, x) \in F'$$

$$\leftrightarrow$$

$$\hat{\delta}(Q_0, x) \cap F \neq \emptyset$$

$$\leftrightarrow$$

$$x \in L(N)$$

Regularity

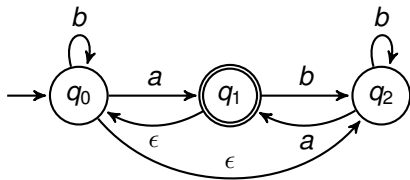
A language L is regular iff there exists an NFA A such that $L = L(A)$

Determinism and Completeness

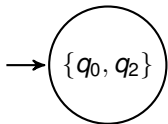
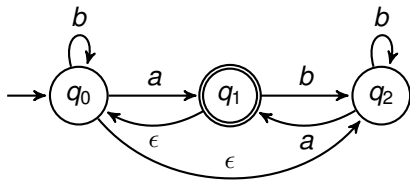
- ▶ Determinism *can* simply mean absence of choices : that is, at any state q , on any symbol a , we do not go to more than one state on a
- ▶ If a state has a missing transition on a symbol a , then it is *incomplete*. This is easy to fix.
- ▶ If a state has multiple outgoing transitions on a , then it needs more work to fix it.
- ▶ What we call a DFA is a deterministic and complete automaton.
- ▶ Going forward, sometimes we use the term deterministic to refer to the absence of choices.

NFA variants to make life easier

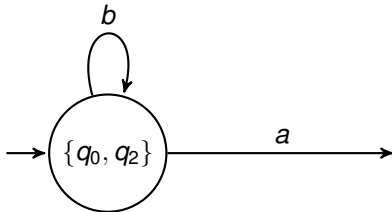
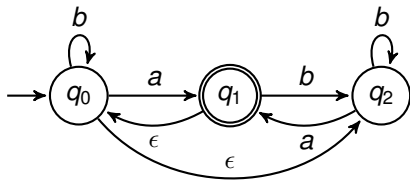
ϵ -NFA



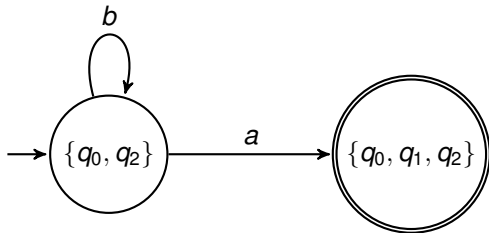
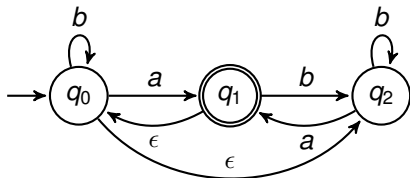
ϵ -NFA



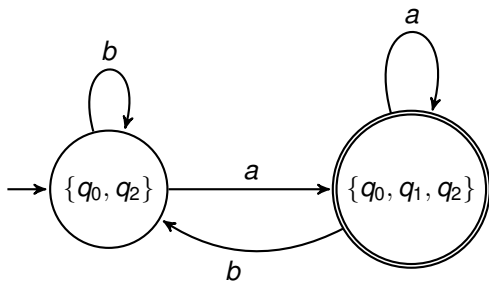
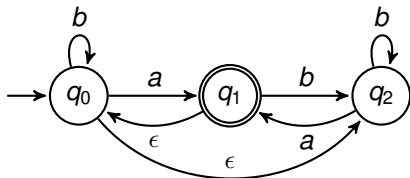
ϵ -NFA



ϵ -NFA



ϵ -NFA

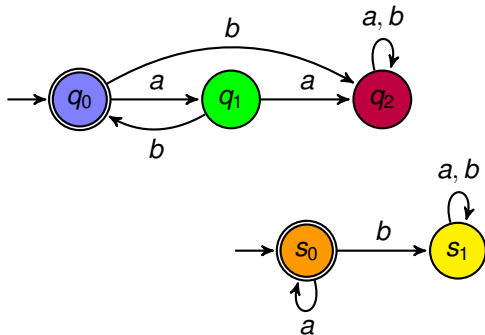


ϵ -NFA and DFA

- ▶ ϵ -close the initial states of the ϵ -NFA to obtain initial state of DFA
- ▶ From a state S , compute $\Delta(S, a)$ and ϵ -close it
- ▶ All states in the DFA are ϵ -closed
- ▶ Final states are those which contain a final state of the ϵ -NFA

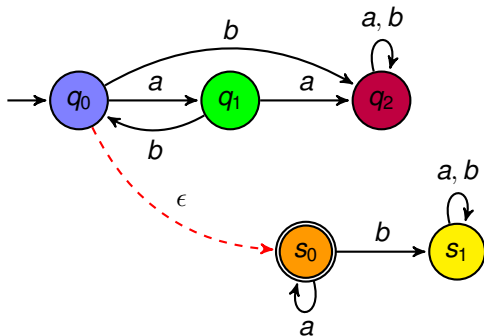
Closure under Concatenation

- ▶ Given regular languages L_1, L_2 , is $L_1.L_2$ regular



Closure under Concatenation

- Given regular languages L_1, L_2 , is $L_1.L_2$ regular?



Formulae to Automaton

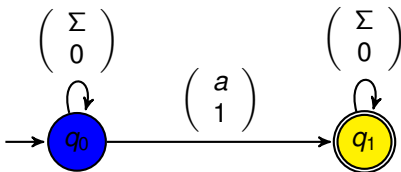
- ▶ FO-definable $\Rightarrow$ regular
- ▶ Given an FO formula φ , construct a DFA A_φ such that $L(\varphi) = L(A_\varphi)$
- ▶ If $L(A_\varphi) = \emptyset$, then φ is unsatisfiable
- ▶ If $L(A_\varphi) \neq \emptyset$, then φ is satisfiable

FO to Regular Languages

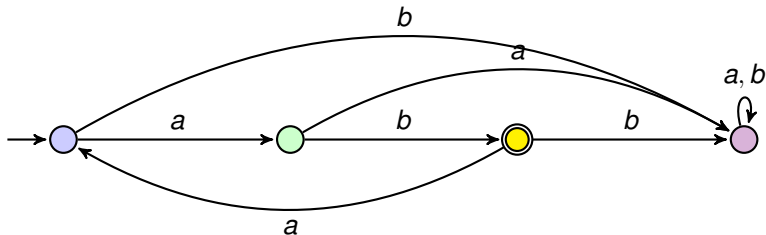
- ▶ Every FO sentence φ over words can be converted into a DFA A_φ such that $L(\varphi) = L(A_\varphi)$.
- ▶ Start with atomic formulae, construct DFA for each of them.
- ▶ Conjunctions, disjunctions, negation of formulae easily handled via union, intersection and complementation of of respective DFA
- ▶ Handling quantifiers?

Atomic Formulae to DFA

- ▶ $Q_a(x)$: All words which have an a . Need to fix a position for x , where a holds.
- ▶ $baab$ satisfies $Q_a(x)$ with assignment $x = 1$ or $x = 2$.
- ▶ Think of this as $\begin{matrix} baab \\ 0010 \end{matrix}$ or $\begin{matrix} baab \\ 0100 \end{matrix}$
- ▶ The first row is over Σ , and the second row captures a possible assignment to x
- ▶ Think of an extended alphabet $\Sigma' = \Sigma \times \{0, 1\}$, and construct an automaton over Σ' .
- ▶ Deterministic, not complete.



Pop Quiz :)



- ▶ What is the language $L(A)$ accepted by this machine A ?
- ▶ Write an FO sentence φ such that $L(\varphi) = L(A)$